Module 4: Functional Requirements Specification
INFO C450 - System Design
Spring 2026
Titus Duncan II

Collaborative Media Management System (CollabSpace), Titus Duncan

1.1 Stakeholders:

1. **Creators (project owners)**: need an organized home for projects, versions, collaborators, and client delivery.
2. **Collaborators (creative teammates)**: need controlled access to upload, version, comment, and track progress.
3. **Clients (guest or account)**: need simple viewing and clear ways to approve or request changes.
4. **Admin / platform maintainer (future scale)**: needs to manage users, reports, and limits.

1.2 Actors and Goals:

1.2.1 Primary Actors

- **Creator**: starts projects, organizes work, manages collaborators, controls "current version," and publishes presentations.
- **Collaborator**: contributes work (assets/versions), updates status, comments, and searches assets—same workflow as creator, but with fewer permissions.
- **Client**: views presentation, uses controlled access, approves a version, or requests changes.

1.2.2 Secondary Actors

- **Admin**: manages users, handles reports, manages platform limits.
- **System**: stores project data, enforces access, maintains "current version," supports search/retrieval.

1.3 Use Cases:

1.3.1 Casual Description + Estimates

Use Case (Creator)	Casual Description	Estimate (pts)
Create project	Start a new project with a title and description so all work stays in one organized place.	2
Manage collaborators	Invite collaborators, assign roles/permissions, and remove access when needed.	4
Set current version	Choose which uploaded version is the official "current" version shown to others.	3
Create presentation page	Build a client-facing page by selecting assets/versions and arranging them for viewing.	4
Create account	Sign up so the creator can access and save projects.	2
Login/Logout	Sign in/out to access projects securely.	2

Upload asset	Add a new asset into a project (file + basic info).	2
Add version	Upload a new version tied to an existing asset and keep version history organized.	3
Update status	Mark progress (draft/in review/approved/changes requested) so the team knows what's going on.	2
Comment on version	Leave feedback connected to the exact version being discussed.	2
Search/filter assets	Search assets by keyword/tags/type to quickly find what's needed.	2
Creator total		**28**

Use Case (Collaborator)	Casual Description	Estimate (pts)
Create account	Sign up to join projects and contribute work.	2
Login/Logout	Sign in/out to access assigned projects.	2
Upload asset	Add a new asset into the shared project (with permissions).	2
Add version	Upload a revision to an existing asset so progress is tracked cleanly.	3
Update status	Update progress labels so the team stays aligned.	2
Comment on version	Give feedback tied to a specific version.	2
Search/filter assets	Find assets fast without digging through folders/links.	2
Collaborator total		**15**

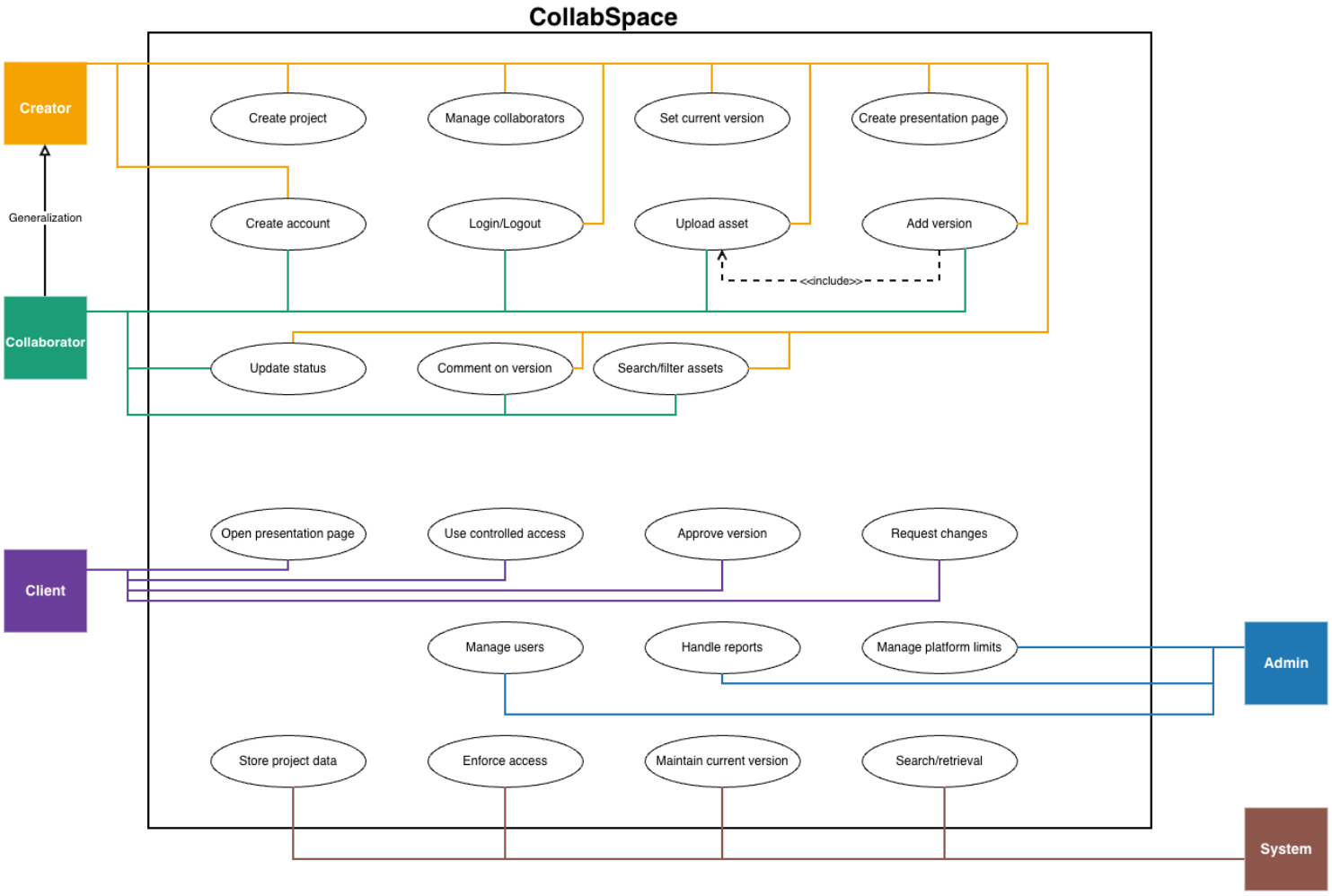
Use Case (Client)	Casual Description	Estimate (pts)
Open presentation page	View the project presentation in a clean, client-friendly format.	2
Use controlled access	Enter through a secure link or account so access stays intentional and limited.	3
Approve version	Approve a specific version so the team knows what is accepted.	3
Request changes	Request changes on a specific version with notes so feedback stays organized.	3
Client total		**11**

Use Case (Admin)	Casual Description	Estimate (pts)
Manage users	View users and change account status when needed (active/suspended/closed).	3
Handle reports	Review and resolve reports (abuse, broken links, inappropriate content).	3
Manage platform limits	Set basic platform limits (storage, project caps) to support scaling.	3
Admin total		**9**

Use Case (System)	Casual Description	Estimate (pts)
Store project data	Save projects, assets, versions, comments, and reviews reliably.	3
Enforce access	Check permissions and link rules so only allowed people can view or act.	3

Maintain current version	Ensure the selected "current" version is always the one shown by default.	3
Search/retrieval	Return search/filter results quickly based on keywords/tags/type.	2
System total		**11**

1.4 Use Case Diagram:

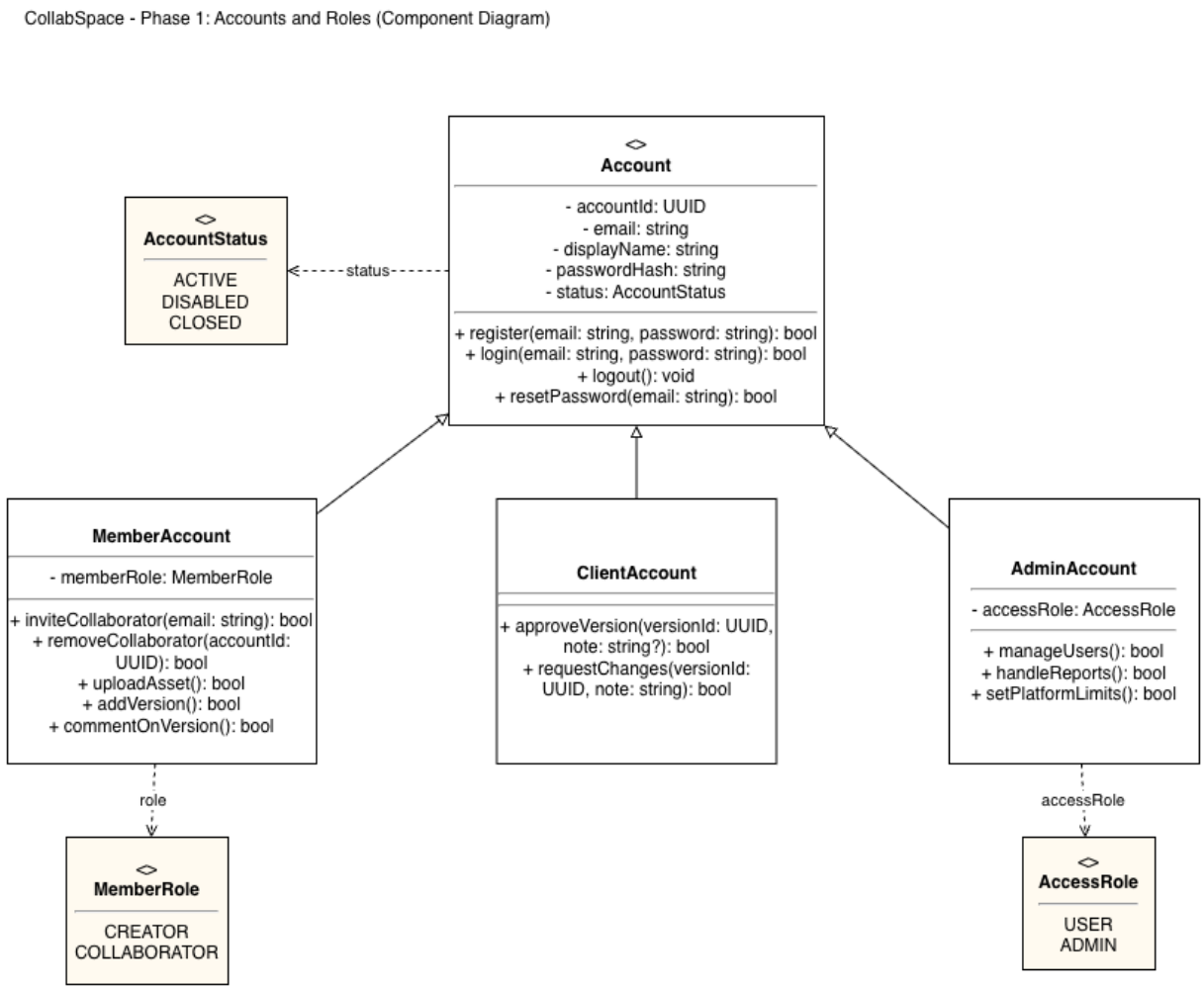


Module 4: Functional Requirements Specification
INFO C450 - System Design
Spring 2026
Titus Duncan II

1.5 Class Diagram:

Diagram 1 (Accounts/Roles)

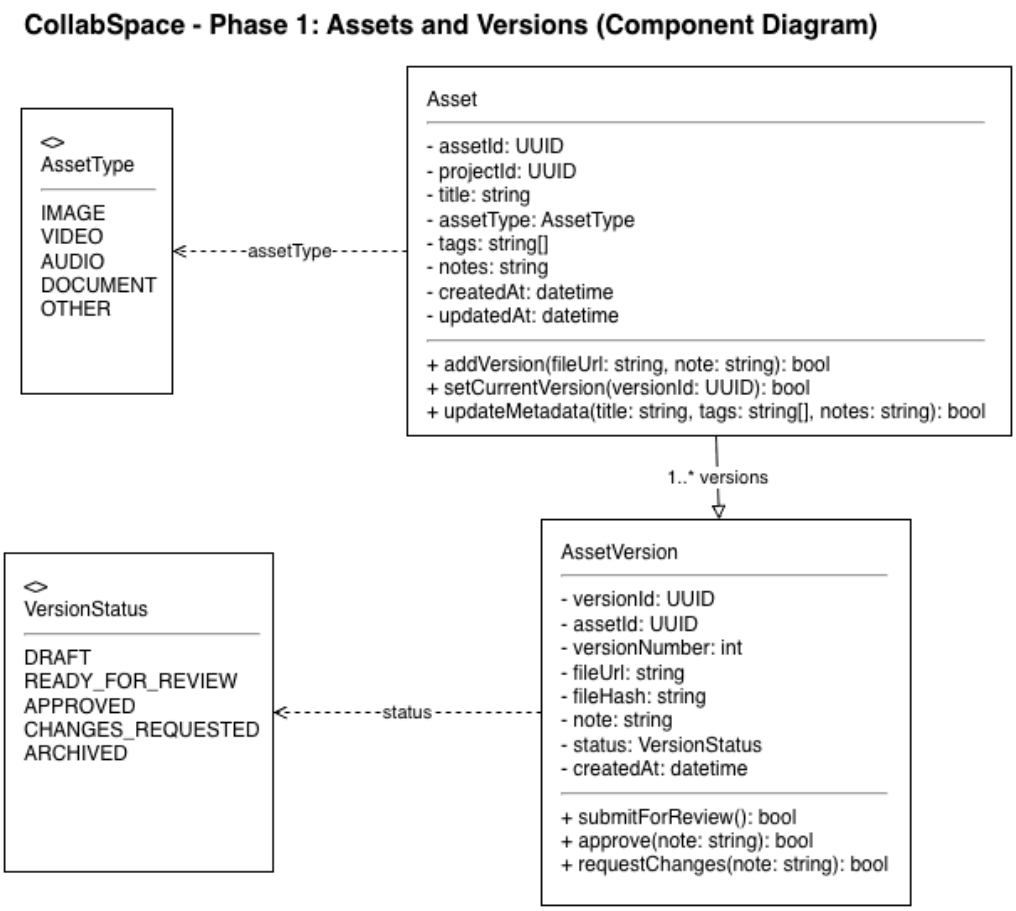
This diagram shows the main Phase 1 classes and the direction of their relationships (navigation). A MemberAccount creates/owns a Project. A Project contains Assets, and each Asset has multiple Versions. A Project publishes a PresentationPage, which uses ControlledAccess to control viewing permissions. A ClientAccount views the PresentationPage and can approve a Version or request changes. An AdminAccount manages Accounts (user administration).



Module 4: Functional Requirements Specification
INFO C450 - System Design
Spring 2026
Titus Duncan II

Diagram 2: Assets + Versions

This component diagram defines how Assets and AssetVersions are structured in CollabSpace, including core attributes, key operations, and the VersionStatus enumeration used to track review state.



Module 4: Functional Requirements Specification
INFO C450 - System Design
Spring 2026
Titus Duncan II

Diagram 3: PresentationPage + ControlledAccess

This component diagram models PresentationPage and its Controlled Access mechanism using an AccessPolicy class and AccessMode enumeration to represent public vs restricted viewing.

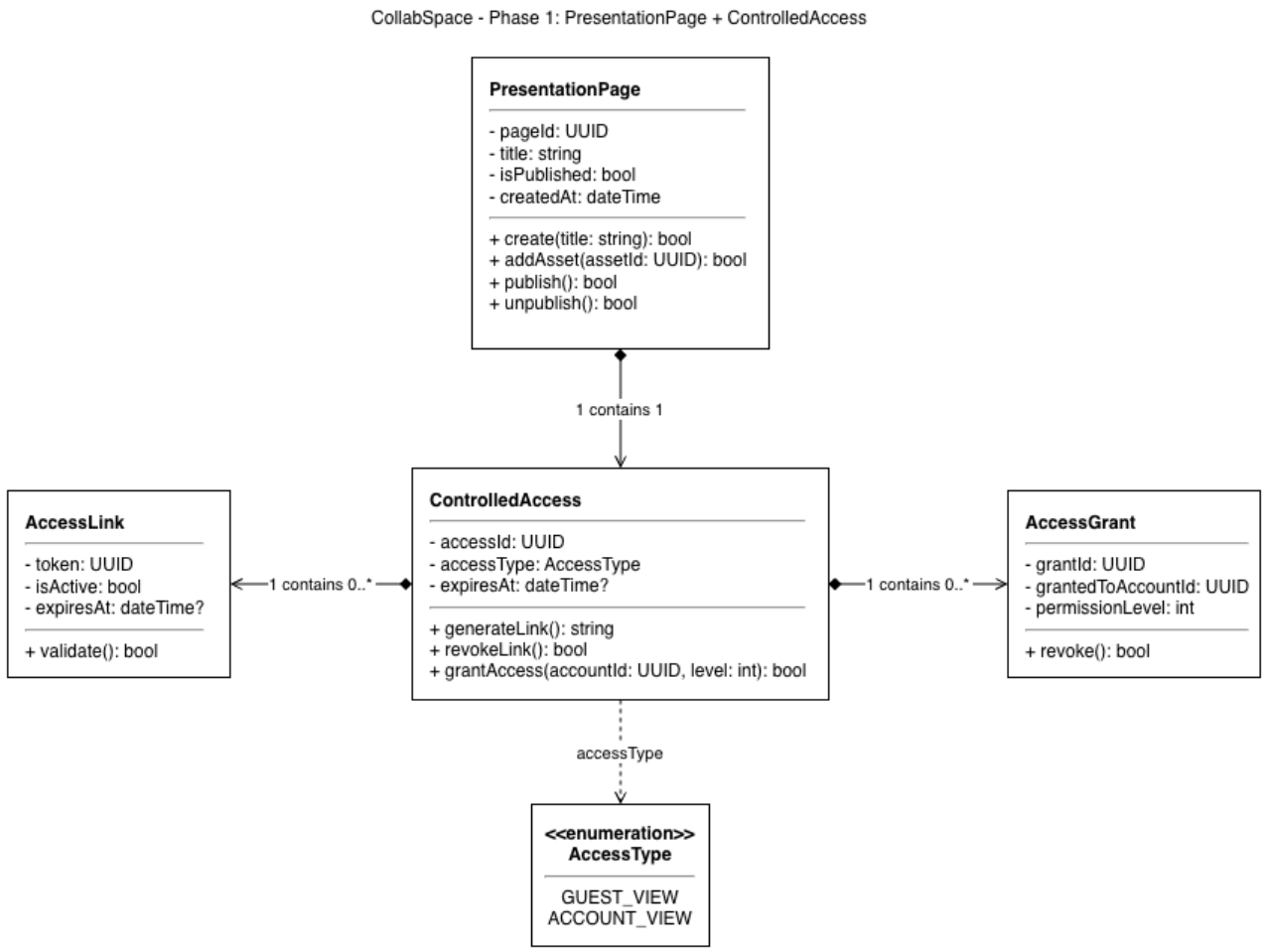
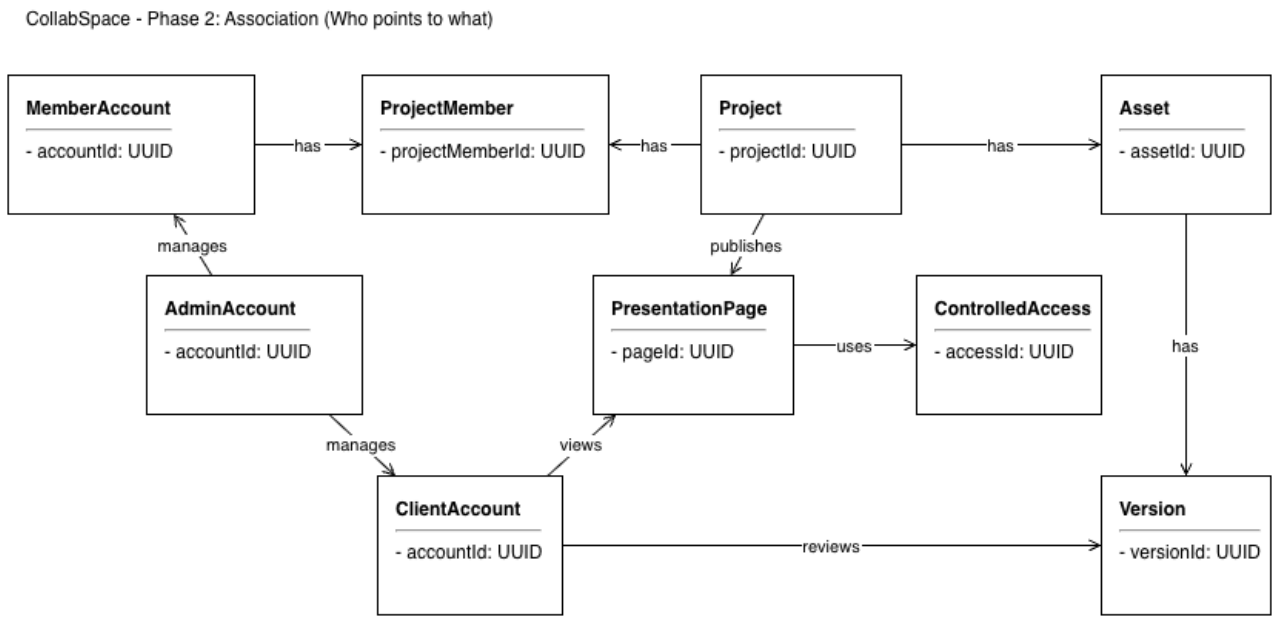


Diagram 4: Associations

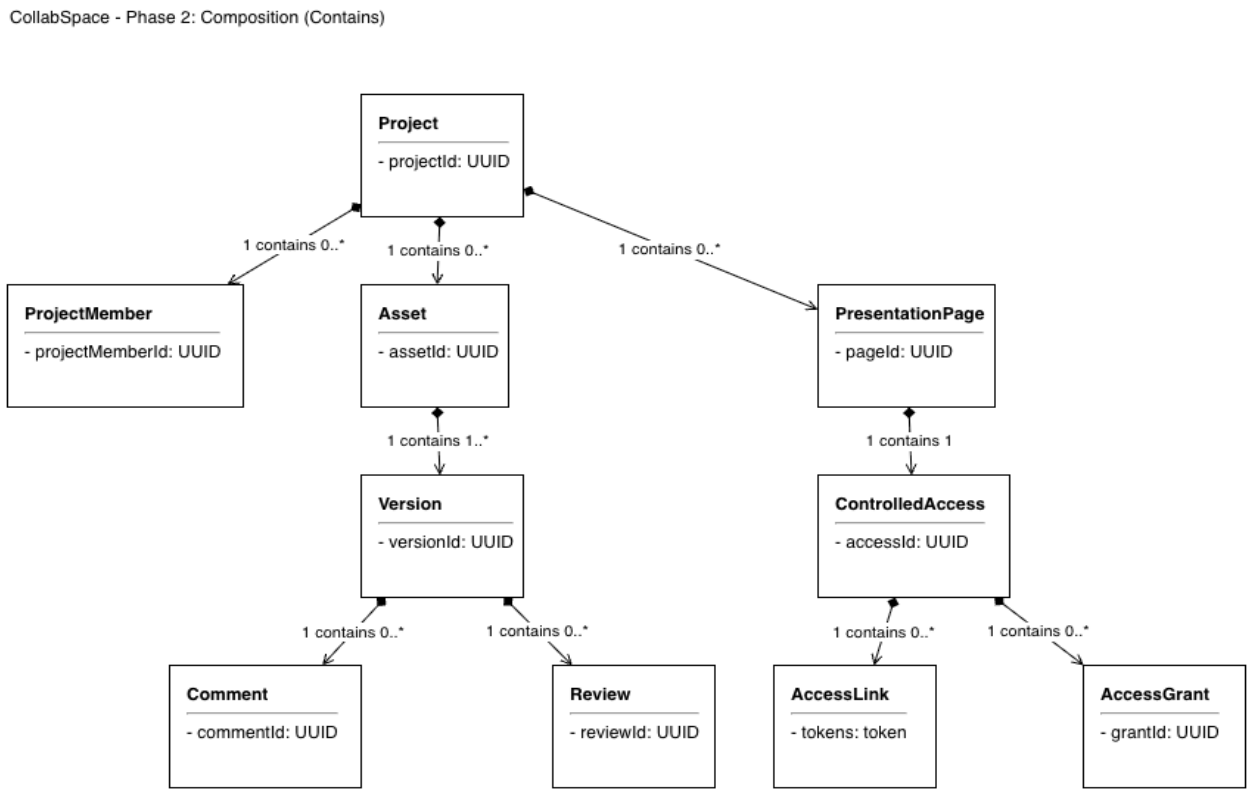
This association diagram shows the direction of references between major CollabSpace classes, clarifying which objects depend on or point to others.



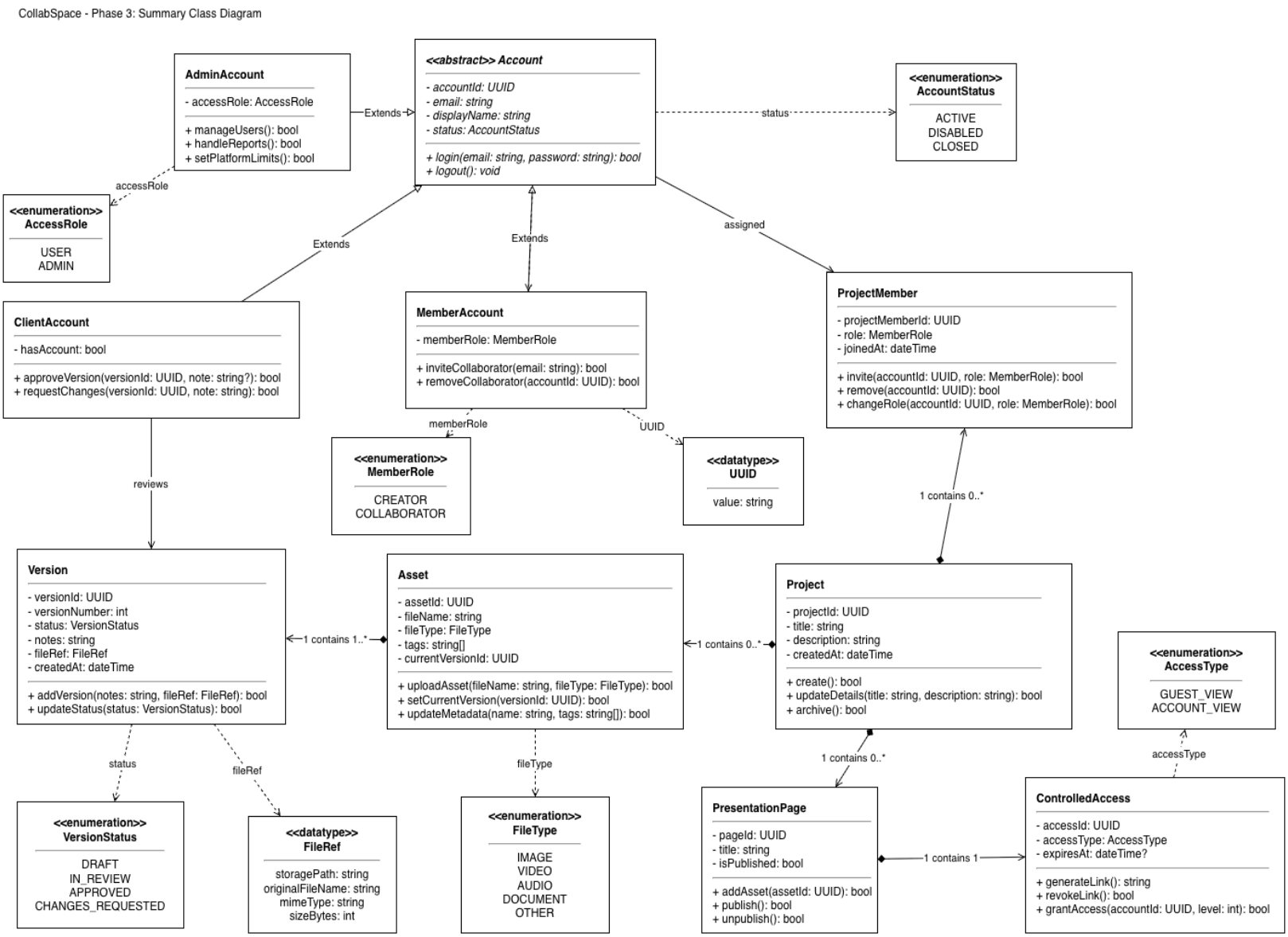
Module 4: Functional Requirements Specification
INFO C450 - System Design
Spring 2026
Titus Duncan II

Diagram 5: Composition

This composition diagram shows which classes are owned/contained by others (whole–part), such as Project containing Assets and Assets containing Versions.



Module 4: Functional Requirements Specification
INFO C450 - System Design
Spring 2026
Titus Duncan II

Diagram 6: Summary Class Diagram

This summary class diagram combines the core CollabSpace components into one unified view, including key attributes/operations, inheritance (Account types), and major associations/compositions across the system.